Deep Research Agent

AI-Powered Research with Hallucination Reduction

Product Overview

Deep Research Agent is an AI-powered research API with built-in hallucination reduction. It gathers information from multiple web sources, synthesizes responses using LLMs, and verifies facts through cross-referencing.

Target Use Cases

- Mobile apps requiring factual research
- Wearable devices with voice queries
- Enterprise knowledge assistants
- Research automation tools

Deep Research Agent - Technical Specification

System Architecture

The system consists of multiple layers working together:

Layer Overview

Layer	Components	Purpose
Client	Web UI, Mobile, API	User interface
Gateway	FastAPI, Validators	Request handling
Cache	Semantic Cache	Cost reduction
Core	Research Agent	Orchestration
Safeguards	Circuit Breaker, Retry	Reliability

Hallucination Reduction System

Technique	Description	Impact
Chain-of-Verification	Draft > Verify > Check > Final	15-25% accuracy gain
Multi-Source Verify	Cross-ref 3-8 sources	Reduces bias
Confidence Scoring	0-1 score per claim	Quantifies reliability
Auto Refinement	Re-gen if conf < 0.7	Improves quality
Source Grounding	LLM uses ONLY sources	Prevents fabrication

Semantic Caching

Feature	Specification
Match Types	Exact + Semantic similarity
Similarity Threshold	0.85 (configurable)
TTL	24 hours (configurable)
Max Entries	1,000 (configurable)
Cost Savings	50-90% on similar queries
Latency	<100ms cache hit vs 2-10s fres

Production Safeguards

Safeguard	Implementation	Purpose
Request Timeout	120s max	Prevent hung requests
Rate Limiting	10 req/min per IP	Prevent abuse
Concurrent Limit	Semaphore (5 max)	Control API costs
Circuit Breaker	Disable after 5 fails	Graceful degradation
Input Validation	Sanitize + limits	Security
Retry w/ Backoff	3 attempts, exp delay	Handle failures

API Specification

Endpoints

Method	Endpoint	Description
GET	/	Web UI
GET	/health	Health check
POST	/research	Perform research
GET	/research	Research (query params)
GET	/research/stream	SSE streaming
GET	/cache/stats	Cache statistics
GET	/docs	Swagger UI

Research Depth Options

Depth	Sources	Speed
quick	3 sources	Fastest (5-15s)
standard	5 sources	Balanced (10-25s)
deep	8+ sources	Thorough (20-45s)

Technology Stack

Component	Technology	Purpose
Framework	FastAPI	Async API server
LLM	Google Gemini 2.5	Response generation
Web Search	DuckDuckGo	Source gathering
HTML Parsing	BeautifulSoup	Content extraction
HTTP Client	httpx (async)	Web requests
Validation	Pydantic	Request models
Retry Logic	Tenacity	Exponential backoff
Vector Math	NumPy	Semantic similarity

Configuration

Environment Variables

```
GOOGLE_API_KEY=your_api_key
API_HOST=0.0.0.0
API_PORT=8000
CACHE_SIMILARITY_THRESHOLD=0.85
CACHE_TTL_HOURS=24
REQUEST_TIMEOUT_SECONDS=120
MAX_CONCURRENT_LLM_CALLS=5
```

Performance Characteristics

Metric	Value
Cache Hit Latency	<100ms
Fresh Query (Quick)	5-15 seconds
Fresh Query (Standard)	10-25 seconds
Fresh Query (Deep)	20-45 seconds
Memory Usage	50-200MB
Max Concurrent LLM	5 calls

Deployment

Quick Start

```
pip install -r requirements.txt
cp env.example .env
# Add GOOGLE_API_KEY to .env
python main.py
# Open http://localhost:8000
```

Production Checklist

- Set specific CORS origins
- Add authentication (API keys/OAuth)
- Use Redis for distributed caching
- Add monitoring/logging
- Configure SSL/TLS